Solution: Pascal's Triangle

Problem Statement:

Write a Recursive Solution to Generate Pascal's Triangle.

Write a recursive function to generate Pascal's Triangle up to a given number of rows. Pascal's Triangle is a triangular array of binomial coefficients, where each number is the sum of the two numbers directly above it.

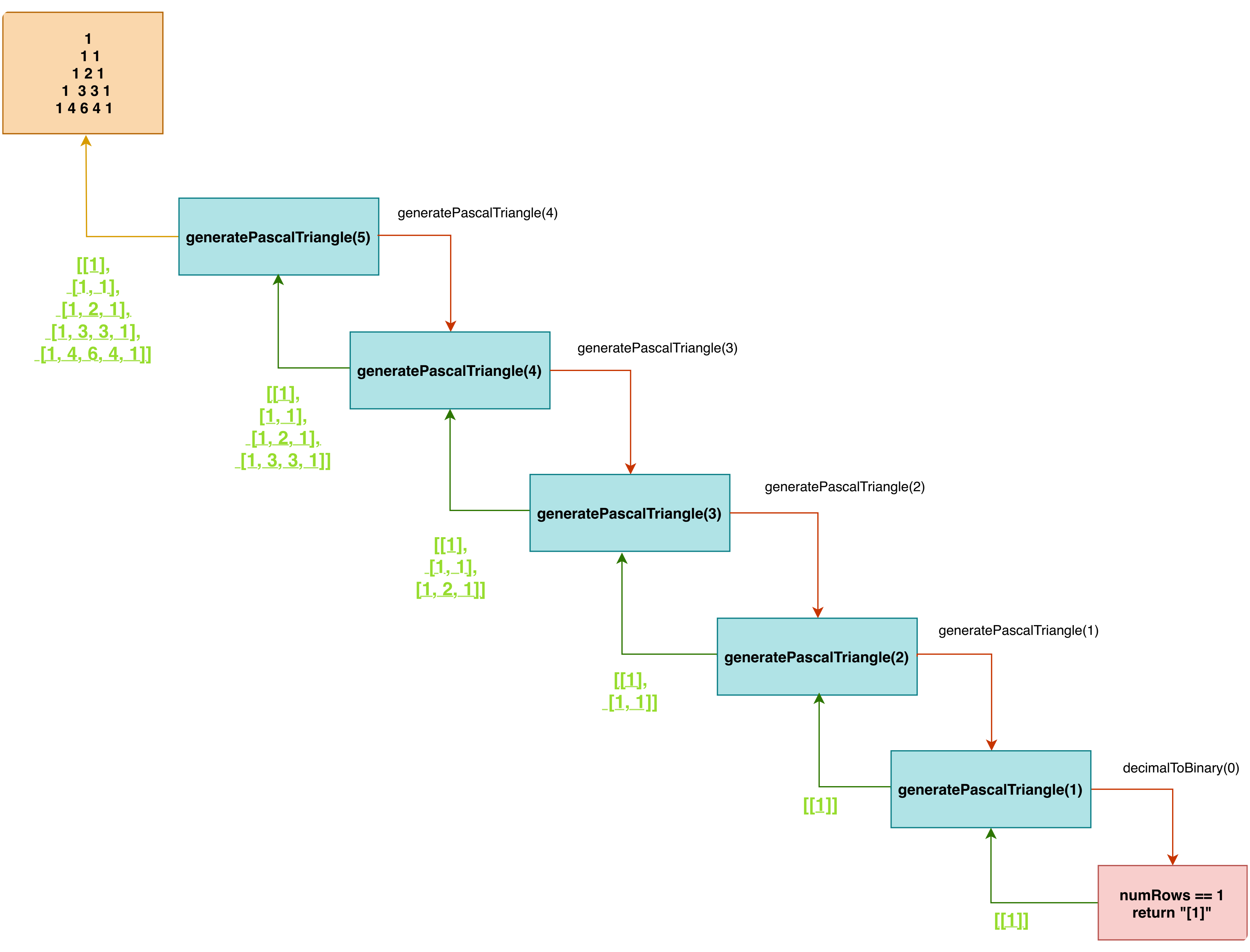
Example:

Sr#	Input	Output	Explanation
1	numRows = 3	[[1],[1,1],[1,2,1]]	The first 3 rows of Pascal's Triangle are [[1],[1,1],[1,2,1]].
2	numRows = 5	[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]	The first 5 rows of Pascal's Triangle are [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]].
3	numRows = 1	[[1]]	The first row of Pascal's Triangle is [[1]].

Solution:

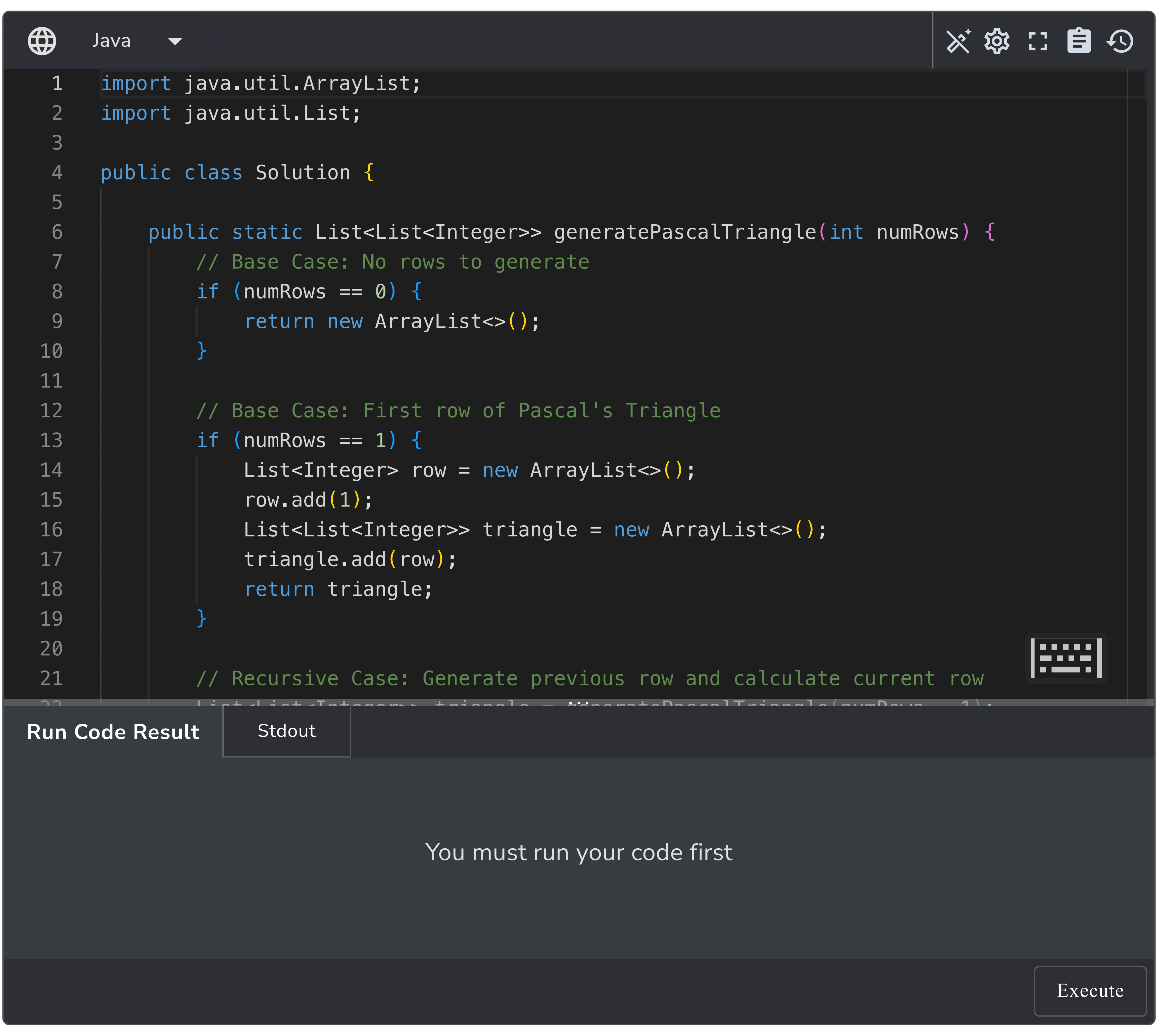
1. Base Case:
    - If the number of rows (numRows) is 0, return an empty list ([]), as there are no rows to generate.
2. Recursive Case:
    - If the number of rows is 1, return a list with a single sublist containing the number 1 ([1]), as it is the first row of Pascal's Triangle.
    - For each row greater than 1, recursively generate the previous row and use it to calculate the current row.
    - The current row is generated by adding adjacent elements from the previous row and appending 1 at the beginning and end.
    - Repeat the process until the desired number of rows is reached.

This approach works by breaking down the problem into smaller subproblems. The base case handles the termination condition when there are no rows to generate. The recursive case leverages the previously generated row to calculate the current row, gradually building Pascal's Triangle row by row.


1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
generatePascalTriangle(5)
generatePascalTriangle(4)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
generatePascalTriangle(4)
generatePascalTriangle(3)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]
generatePascalTriangle(3)
generatePascalTriangle(2)
[[1], [1, 1], [1, 2, 1]]
generatePascalTriangle(2)
generatePascalTriangle(1)
[[1], [1, 1]]
generatePascalTriangle(1)
decimalToBinary(0)
[[1]]
numRows == 1
return "[1]"
[[1]]

Code

Here is the code for this algorithm:



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {

    public static List<List<Integer>> generatePascalTriangle(int numRows) {
        // Base Case: No rows to generate
        if (numRows == 0) {
            return new ArrayList<>();
        }

        // Base Case: First row of Pascal's Triangle
        if (numRows == 1) {
            List<Integer> row = new ArrayList<>();
            row.add(1);
            List<List<Integer>> triangle = new ArrayList<>();
            triangle.add(row);
            return triangle;
        }

        // Recursive Case: Generate previous row and calculate current row
```

Run Code Result
Stdout

You must run your code first

Time Complexity:

The time complexity of this algorithm is $O(n^2)$, where n is the number of rows. This is because we generate each row based on the previous row, and the number of elements in each row increases linearly with the row number.

Space Complexity:

The space complexity of this algorithm is $O(n^2)$, where n is the number of rows. This is because we store the Pascal's Triangle as a list of lists, with each sublist representing a row. The size of the triangle grows quadratically with the number of rows.

← Previous
8. Pascal's Triangle
Next →
9. Binary Search
☐ Mark as Completed